SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: *Apply object-oriented concepts and software development life cycle models to design structured and modular software systems.* *(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr MANIMAARAN

QUESTIONS: 6 **Total Marks:30**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I Dhaanush Rajan*(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Short Answer Test

1.Suppose a software system is developed for a **Smart Banking System** where customers can open accounts, transfer funds, and pay utility bills.

(a) Explain the following object-oriented concepts:

- Class
- Object
- Message Passing
- Encapsulation
- Polymorphism

(b) Describe the following object relationships:

- Association
- Aggregation
- Composition

(c) Explain how these concepts improve software reusability and maintainability in a banking system.

(5 Marks)

ANSWER

(a) Explain the following Object-Oriented Concepts

1. Class

A **Class** is a blueprint or template used to create objects. It defines the attributes (data) and methods (functions) that an object will have.

Example (Smart Banking System):

- **Class:** Customer
- **Attributes:** CustomerID, Name, Address, PhoneNumber
- **Methods:** openAccount(), updateProfile(), viewBalance()

2. Object

An **Object** is an instance of a class. It represents a real-world entity with its own data and behavior.

Example:

- **Object 1:** Customer – Rahul

- **Object 2:** Customer – Priya

Both are objects of the **Customer** class but contain different information.

3. Message Passing

Message Passing is the communication between objects by calling each other's methods.

Example:
When a customer transfers ₹10,000:

- The **Customer** object sends a request to the **BankAccount** object.
- The **BankAccount** object processes the transfer and updates the balance.

This interaction between objects is called **message passing**.

4. Encapsulation

Encapsulation is the process of combining data and methods into a single unit (class) while restricting direct access to sensitive data.

Example:
In the **BankAccount** class:

- **Private Data:** accountNumber, balance
- **Public Methods:** deposit(), withdraw(), checkBalance()

The balance cannot be changed directly. It can only be modified through authorized methods.

Advantages:

- Protects sensitive banking data
- Prevents unauthorized access
- Improves security

5. Polymorphism

Polymorphism means "one interface, many forms." The same method performs different actions depending on the object.

Example:

Method:

calculateInterest()

Different account types:

- Savings Account → 4% interest
- Current Account → 0% interest

- Fixed Deposit → 7% interest

The same method behaves differently for each account type.

(b) Object Relationships

1. Association

Association is a relationship where two objects communicate with each other but remain independent.

Example:

- Customer ↔ BankAccount

A customer owns one or more bank accounts.

Representation:

Customer -------- BankAccount

2. Aggregation

Aggregation is a **"has-a"** relationship where one object contains another object, but both can exist independently.

Example:

- Bank has many Customers.

Even if the bank branch closes, customer records may still exist in another branch.

Representation:

Bank ◇-------- Customer

(Hollow diamond)

3. Composition

Composition is a strong **"part-of"** relationship where the child object cannot exist without the parent object.

Example:

- BankAccount contains Transaction History.

If the bank account is permanently deleted, all its transaction records are also deleted.

Representation:

BankAccount ◆-------- Transaction

(Filled diamond)

(c) How These Concepts Improve Software Reusability and Maintainability

1. Reusability

- Classes such as **Customer**, **Account**, and **Transaction** can be reused in different banking modules.
- Inheritance and polymorphism reduce duplicate code.
- Existing classes can be extended without rewriting them.

Example:
The **Account** class can be reused to create:

- Savings Account
- Current Account
- Loan Account

2. Maintainability

- Encapsulation hides internal implementation details.
- Changes made in one class have minimal impact on other classes.
- Bugs can be fixed easily because each class handles a specific responsibility.
- Modular design makes testing and debugging simpler.

3. Flexibility

New banking services such as:

- Mobile Banking
- UPI Payments
- Credit Card Management
- Loan Processing

can be added by creating new classes without affecting existing code.

4. Security

Encapsulation protects sensitive information like:

- Account Balance
- PIN
- Password
- Customer Details

Only authorized methods can access or modify these values.

5. Scalability

As the banking system grows, new features can be added with minimal changes, making the software easier to upgrade and maintain.

2.Consider developing a **Smart Healthcare Management System**.

(a) Explain the following object-oriented design principles:

- Abstraction
- Encapsulation
- Modularity

(b) Describe the following SOLID principles:

- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)

(c) Explain how these principles improve software quality and flexibility.

(5 Marks)

ANSWER

(a) Object-Oriented Design Principles

1. Abstraction

Abstraction means hiding unnecessary implementation details and showing only the essential features to the user.

Example:
In a Smart Healthcare Management System, a **Doctor** can view patient records and prescribe medicines without knowing how the data is stored in the database.

Advantages:

- Reduces complexity.
- Improves security by hiding internal details.
- Makes the system easier to use.

2. Encapsulation

Encapsulation is the process of combining data and methods into a single class while restricting direct access to sensitive data.

Example:
The **Patient** class contains private attributes such as:

- Patient ID
- Medical History
- Diagnosis

These details can only be accessed or modified using methods like getMedicalHistory() or updateDiagnosis().

Advantages:

- Protects patient information.
- Prevents unauthorized access.
- Improves data integrity.

3. Modularity

Modularity means dividing the software into smaller, independent modules, each responsible for a specific task.

Example Modules:

- Patient Management
- Appointment Scheduling
- Billing System
- Pharmacy Management
- Laboratory Reports

Each module can be developed, tested, and maintained independently.

Advantages:

- Easier maintenance.
- Faster development.
- Simplifies debugging and testing.

(b) SOLID Principles

1. Liskov Substitution Principle (LSP)

The **Liskov Substitution Principle** states that a subclass should be able to replace its parent class without affecting the correctness of the program.

Example:

Parent Class:

- Doctor

Child Classes:

- GeneralPhysician
- Cardiologist
- Dermatologist

Any child class can be used wherever a **Doctor** object is expected without changing the program's behavior.

Benefit:

- Promotes code reusability.
- Ensures consistent behavior.
- Reduces errors caused by incorrect inheritance.

2. Interface Segregation Principle (ISP)

The **Interface Segregation Principle** states that a class should not be forced to implement methods it does not need. Instead of one large interface, use multiple smaller, specific interfaces.

Example:

Instead of one interface:

HealthcareService
- prescribeMedicine()
- performSurgery()
- generateBill()

Create smaller interfaces:

- PrescriptionService
- SurgeryService
- BillingService

A **Pharmacist** only implements PrescriptionService, while the **Billing Department** implements BillingService.

Benefit:

- Avoids unnecessary methods.
- Makes the code cleaner.
- Improves maintainability.

(c) How These Principles Improve Software Quality and Flexibility

1. Better Software Quality

- Abstraction hides unnecessary details, making the system easier to understand.
- Encapsulation protects sensitive patient data.
- Modularity reduces complexity and improves organization.

2. Improved Flexibility

- New healthcare services such as Telemedicine or Online Consultation can be added without affecting existing modules.
- LSP allows new doctor types to be introduced without changing existing code.
- ISP enables adding new interfaces without forcing changes to unrelated classes.

3. Easier Maintenance

- Independent modules can be updated separately.
- Bugs can be fixed without affecting the entire system.
- Code is easier to test and debug.

4. Increased Reusability

- Existing classes and interfaces can be reused in different healthcare applications.
- Common modules like billing and appointment scheduling can be integrated into other medical systems.

5. Enhanced Security

- Encapsulation ensures that confidential information such as medical records and prescriptions is accessed only through authorized methods.

3.Suppose a software company is developing an **Online Food Delivery System**.

(a) Explain the phases of the Software Development Life Cycle (SDLC):

- Planning
- Requirement Analysis
- Design
- Development
- Testing
- Maintenance

(b) Describe the purpose of each SDLC phase.

(c) Explain how SDLC contributes to delivering reliable software.

(5 Marks)

ANSWER

(a) Phases of the Software Development Life Cycle (SDLC)

1. Planning

Planning is the first phase of SDLC where the project's objectives, scope, budget, timeline, and feasibility are determined.

Example:
The software company plans to develop an Online Food Delivery System with features such as user registration, restaurant listing, food ordering, online payment, and order tracking.

2. Requirement Analysis

In this phase, the requirements of customers and stakeholders are gathered, analyzed, and documented.

Example Requirements:

- Customer registration and login
- Search restaurants and menus
- Place food orders
- Online payment
- Live order tracking
- Delivery partner management
- Admin dashboard

3. Design

The design phase defines the system architecture, database structure, user interface, and software components.

Example:

- Database for customers, restaurants, orders, and payments
- User-friendly mobile/web interface
- Architecture connecting customers, restaurants, and delivery partners

4. Development

During this phase, developers write the program code based on the design specifications.

Example:
Developers create modules for:

- User Authentication
- Restaurant Management
- Order Processing
- Payment Gateway
- Delivery Tracking

5. Testing

Testing verifies that the software works correctly and is free from defects.

Types of Testing:

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing (UAT)

Example:
Verify that food orders are placed successfully, payments are processed correctly, and order tracking updates in real time.

6. Maintenance

After deployment, the software is monitored, updated, and improved to fix bugs and add new features.

Example:

- Fix payment issues
- Improve app performance
- Add new payment methods
- Introduce features like coupon codes and loyalty rewards

(b) Purpose of Each SDLC Phase

SDLC Phase	Purpose
Planning	Defines project goals, scope, budget, schedule, and feasibility.
Requirement Analysis	Collects and documents user and business requirements.
Design	Creates the system architecture, database, and user interface.
Development	Converts the design into a working software application through coding.
Testing	Identifies and fixes defects to ensure the software functions correctly.
Maintenance	Provides updates, bug fixes, performance improvements, and new features after deployment.

(c) How SDLC Contributes to Delivering Reliable Software

1. Clear Planning

Proper planning ensures the project has well-defined goals, realistic timelines, and efficient resource allocation.

2. Accurate Requirements

Requirement analysis ensures the software meets customer needs and reduces misunderstandings.

3. Well-Structured Design

A good design creates a scalable, secure, and efficient system that is easier to develop and maintain.

4. Quality Development

Coding according to design standards improves consistency and reduces errors.

5. Thorough Testing

Testing detects and fixes bugs before release, improving reliability, security, and performance.

6. Continuous Maintenance

Regular updates keep the software secure, compatible with new technologies, and responsive to changing user needs.

Advantages of SDLC

- Produces high-quality and reliable software.
- Reduces development risks and costs.

- Improves communication among stakeholders.
- Simplifies project management.
- Makes software easier to maintain and enhance.
- Increases customer satisfaction.

4.Consider a project for developing a **Smart Agriculture Monitoring System**, where customer requirements change regularly.

(a) Explain the following SDLC models:

- Prototype Model
- Spiral Model

- Agile Model
- DevOps Model

(b) Compare the Prototype Model and Agile Model.

(c) Recommend the most suitable SDLC model for this project with justification.

(5 Marks)

ANSWER

(a) Explain the Following SDLC Models

1. Prototype Model

The **Prototype Model** involves developing a preliminary version (prototype) of the software to understand customer requirements. The prototype is reviewed by users, and feedback is used to improve the system before the final product is developed.

Example:
A prototype of the Smart Agriculture Monitoring System may include:

- Crop monitoring dashboard
- Soil moisture display
- Weather information

Farmers provide feedback, and the system is modified accordingly.

Advantages:

- Better understanding of user requirements.
- Early user involvement.
- Reduces requirement misunderstandings.

2. Spiral Model

The **Spiral Model** combines iterative development with risk analysis. The project is developed in multiple cycles (spirals), and risks are identified and addressed in each cycle.

Phases in each Spiral:

- Planning
- Risk Analysis
- Development
- Evaluation

Example:
Before adding an AI-based irrigation feature, the team evaluates technical risks and then develops and tests the feature.

Advantages:

- Excellent risk management.
- Suitable for large and complex projects.
- Flexible to requirement changes.

3. Agile Model

The **Agile Model** develops software in small iterations called **sprints**. Customers provide continuous feedback after each sprint, allowing the team to adapt quickly to changing requirements.

Example:
Sprint 1:

- Farmer Registration
- Login

Sprint 2:

- Soil Moisture Monitoring

Sprint 3:

- Weather Forecast

Sprint 4:

- Automatic Irrigation Control

Each sprint delivers a working feature.

Advantages:

- Quickly adapts to changing requirements.
- Frequent customer feedback.
- Faster delivery of working software.
- Improves customer satisfaction.

4. DevOps Model

The **DevOps Model** combines software development (Dev) and IT operations (Ops) to automate building, testing, deployment, and monitoring.

Example:
Whenever developers update the irrigation module:

- Code is automatically tested.
- The new version is deployed.
- The system is continuously monitored.

Advantages:

- Faster software delivery.
- Continuous Integration (CI) and Continuous Deployment (CD).
- Improved collaboration between development and operations teams.
- Faster bug fixes and updates.

(b) Comparison: Prototype Model vs Agile Model

Feature	Prototype Model	Agile Model
Objective	Understand requirements through a prototype	Deliver software in small working increments
Requirement Changes	Handled after prototype feedback	Easily handled in every sprint
Customer Involvement	Mainly during prototype evaluation	Continuous throughout the project
Development Process	Prototype first, then final system	Incremental and iterative development
Delivery	Final product after prototype approval	Working software delivered after each sprint
Best For	Unclear or incomplete requirements	Frequently changing requirements

(c) Recommended SDLC Model

Recommended Model: Agile Model

The **Agile Model** is the most suitable choice for the **Smart Agriculture Monitoring System** because customer requirements change regularly.

Justification

1. **Supports Frequent Requirement Changes**
    - Farmers may request new features such as pest detection, fertilizer recommendations, or weather alerts.
    - Agile allows these changes to be added in future sprints without restarting the project.
2. **Continuous Customer Feedback**
    - Farmers and agricultural experts can review each sprint and suggest improvements.
    - This ensures the system meets real user needs.
3. **Faster Delivery**
    - Useful features are released early instead of waiting for the entire system to be completed.
4. **Improved Software Quality**
    - Regular testing in each sprint helps identify and fix defects early.
5. **Better Flexibility**

    - New technologies like IoT sensors, drones, and AI-based crop monitoring can be integrated easily.

Advantages of Agile for Smart Agriculture

- Adapts quickly to changing customer requirements.
- Delivers working software faster.
- Encourages regular customer feedback.
- Improves software quality through continuous testing.
- Easier to maintain and upgrade.
- Reduces project risk.

5.Suppose a software analyst is designing a **Smart Transport Management System**.

(a) Explain the importance of UML in object-oriented software development.

(b) Describe the following UML diagrams:

- State Diagram

- Component Diagram
- Deployment Diagram
- Package Diagram

(c) Explain how UML models improve communication among stakeholders.

(5 Marks)

ANSWER

(a) Importance of UML in Object-Oriented Software Development

Unified Modeling Language (UML) is a standardized visual modeling language used to analyze, design, and document object-oriented software systems. It helps developers and stakeholders understand the structure and behavior of the system before coding begins.

Importance of UML

- Provides a clear visual representation of the system.
- Helps understand system requirements.
- Improves communication among developers, clients, and stakeholders.
- Reduces design errors before implementation.
- Makes software easier to maintain and enhance.
- Serves as proper documentation for future development.

Example:
In a **Smart Transport Management System**, UML is used to model entities such as **Passenger, Driver, Vehicle, Route, Ticket, and Payment**, and their interactions.

(b) UML Diagrams

1. State Diagram

A **State Diagram** represents the different states of an object and the transitions between those states during its lifecycle.

Example:
For a **Ticket** object, the states may be:

- Available
- Booked
- Confirmed
- Completed
- Cancelled

Purpose:

- Shows how an object changes from one state to another.
- Models dynamic behavior.
- Helps understand the lifecycle of objects.

---

2. Component Diagram

A **Component Diagram** illustrates the software components of a system and the relationships between them.

Example Components:

- User Interface
- Booking Module
- Payment Module
- Vehicle Management Module
- Database

Purpose:

- Represents the architecture of the software.
- Shows dependencies among components.
- Helps developers understand the overall system structure.

---

3. Deployment Diagram

A **Deployment Diagram** shows how software components are deployed on hardware devices and connected through a network.

Example:
The Smart Transport Management System may include:

- Customer Mobile Application
- Web Server
- Application Server
- Database Server

Purpose:

- Shows the physical deployment of the system.
- Illustrates communication between hardware devices.
- Assists in deployment and infrastructure planning.

---

4. Package Diagram

A **Package Diagram** groups related classes into packages and shows the dependencies among them.

Example Packages:

- User Management
- Booking Management
- Payment Management
- Vehicle Management
- Reports

Purpose:

- Organizes large systems into manageable sections.
- Reduces system complexity.
- Improves maintainability and code organization.

(c) How UML Models Improve Communication Among Stakeholders

1. Better Understanding

UML diagrams visually represent the system, making it easier for developers, managers, testers, and clients to understand the design.

2. Clear Requirements

Stakeholders can review UML models to ensure the system meets business and user requirements before development begins.

3. Improved Team Collaboration

Developers, designers, testers, and project managers use UML diagrams as a common reference, improving coordination and reducing misunderstandings.

4. Early Error Detection

Design issues can be identified during the modeling phase, reducing development costs and saving time.

5. Easier Documentation and Maintenance

UML diagrams serve as permanent documentation, making future maintenance, upgrades, and enhancements easier.

Advantages of UML

- Standardized modeling language.
- Improves communication among stakeholders.
- Simplifies complex software design.
- Reduces design errors.
- Supports software maintenance and reusability.
- Enhances overall software quality.

6.Consider developing a **Smart Inventory Management System**.

(a) Explain the concept of modular software design.

(b) Compare the procedural programming approach with the object-oriented programming approach.

(c) Explain how modularity improves software scalability, maintainability, and reusability.

(5 Marks)

ANSWER

(a) Explain the Concept of Modular Software Design

Modular Software Design is a software development approach in which a large system is divided into smaller, independent modules. Each module performs a specific function and can be developed, tested, maintained, and updated independently.

Example (Smart Inventory Management System):
The system can be divided into the following modules:

- Product Management
- Inventory Management
- Supplier Management
- Order Management
- Billing and Payment
- Report Generation

Each module works independently but communicates with other modules to perform the overall system functions.

Advantages:

- Reduces system complexity.
- Simplifies development and testing.
- Makes maintenance easier.
- Encourages code reuse.

(b) Comparison: Procedural Programming vs Object-Oriented Programming

Feature	Procedural Programming	Object-Oriented Programming (OOP)
Approach	Divides the program into functions or procedures	Divides the program into classes and objects
Focus	Functions and procedures	Data and objects
Data Security	Data is generally accessible by all functions	Data is protected using encapsulation
Reusability	Limited code reuse	High code reuse through inheritance and polymorphism
Maintenance	Difficult for large applications	Easier due to modular design
Scalability	Less suitable for large systems	Highly suitable for large and complex systems
Example	C	Java, C++, Python

(c) How Modularity Improves Software Scalability, Maintainability, and Reusability

1. Scalability

Modularity allows new features or modules to be added without affecting existing modules.

Example:
A **Barcode Scanning Module** or **AI Stock Prediction Module** can be added to the inventory system without modifying the Product Management module.

2. Maintainability

Since each module performs a specific task, developers can fix bugs or update one module without impacting the entire system.

Example:
If there is an issue in the **Billing Module**, only that module needs to be updated while other modules continue to function normally.

3. Reusability

Modules developed for one project can be reused in other software applications, reducing development time and cost.

Example:
The **User Authentication Module** developed for the inventory system can also be reused in a Sales Management System or Hospital Management System.

4. Easier Testing

Each module can be tested independently, making it easier to identify and fix defects before integrating the complete system.

5. Improved Team Collaboration

Different development teams can work on separate modules simultaneously, increasing productivity and reducing project completion time.

Advantages of Modular Software Design

- Reduces software complexity.
- Improves scalability.
- Simplifies maintenance and debugging.
- Promotes code reusability.
- Supports parallel development by multiple teams.
- Increases software reliability and quality.

RUBRICS FOR CALCULATION

SHORT ANSWER TEST

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	20%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps